THEORY REPORT

Assignment 3

Title: Use ADB to Collect Android Device Information and Identify Exposed Debugging Features and Potential Security Risks

1. Aim

To use **Android Debug Bridge (ADB)** to collect Android device information and analyze debugging configurations, exposed interfaces, and potential security risks.

2. Objectives

The objectives of this practical are:

1. To understand the purpose and functionality of ADB.
2. To establish communication between a computer and an Android device using ADB.
3. To collect basic Android device and system information.
4. To identify whether debugging features are enabled.
5. To analyze potential security risks associated with exposed debugging interfaces.
6. To understand secure practices for managing Android debugging features.

3. Introduction

Android Debug Bridge (ADB) is a command-line tool that allows communication between a computer and an Android device or Android emulator. It is primarily used by Android developers and security professionals for application testing, debugging, device management, and troubleshooting.

ADB provides several capabilities, including executing shell commands, transferring files, installing applications, viewing logs, and obtaining system information. These capabilities are useful during development but can become a security concern when debugging interfaces are unnecessarily enabled or exposed.

For example, **USB debugging** allows a connected computer to communicate with an Android device through ADB. If an unauthorized computer obtains debugging authorization, it may gain significant access to the device. Similarly, improperly exposed debugging services over a network can increase the attack surface.

Therefore, ADB can be used not only for device administration but also for security assessment of an Android device.

4. Android Debug Bridge Architecture

ADB follows a client-server architecture consisting of three major components:

4.1 ADB Client

The ADB client runs on the computer and is used to issue commands.

Example:

adb devices

4.2 ADB Server

The ADB server runs in the background on the computer and manages communication between the client and Android devices.

4.3 ADB Daemon

The **adbd** process runs on the Android device and receives and executes ADB commands according to the device's security configuration.

The basic communication flow is:

Computer → ADB Client → ADB Server → ADB Daemon → Android Device

5. Establishing ADB Connection

Before collecting information, ADB must be installed on the computer and USB debugging must be enabled on the test Android device.

The following command checks whether ADB is available:

adb version

To identify connected devices:

adb devices

A connected and authorized device may appear as:

List of devices attached

XXXXXXXX device

The device should be used only with appropriate authorization.

6. Collecting Android Device Information

ADB provides several commands for collecting system information.

6.1 Device Model

adb shell getprop ro.product.model

This command displays the device model.

6.2 Android Version

adb shell getprop ro.build.version.release

This identifies the Android operating system version.

6.3 Security Patch Level

adb shell getprop ro.build.version.security_patch

The security patch level helps determine how recently Android security updates were applied.

6.4 Device Properties

adb shell getprop

This command displays a large collection of Android system properties.

6.5 Battery Information

adb shell dumpsys battery

This provides information about battery state, charging status, and related parameters.

6.6 Running Services and System Information

adb shell dumpsys

The dumpsys utility provides diagnostic information from Android system services.

7. Identifying Debugging Features

ADB can be used to examine whether certain debugging-related configurations are enabled.

For example:

adb shell settings get global adb_enabled

A value indicating that ADB is enabled can be investigated further in the context of the test device's configuration.

The developer options state can also be examined using:

adb shell settings get global development_settings_enabled

These settings should be interpreted carefully because the exact behavior and available settings can vary between Android versions and device manufacturers.

8. Potential Security Risks

Debugging functionality is useful for development but can introduce security risks if it is improperly configured.

8.1 Unauthorized ADB Access

If USB debugging is enabled and an attacker gains physical access to the device, they may attempt to establish an ADB connection.

Risk: Unauthorized access to device functionality and diagnostic information.

Protection: Keep USB debugging disabled when it is not required and authorize only trusted computers.

8.2 ADB over Network

Some Android configurations support ADB communication over a network.

Risk: Network-exposed debugging interfaces can increase the attack surface, particularly on untrusted networks.

Protection: Avoid exposing ADB over unsecured networks and disable network debugging when it is not required.

8.3 Developer Options

Developer Options provide advanced functionality intended primarily for development and testing.

Risk: Incorrect configuration may expose additional debugging or testing functionality.

Protection: Disable unnecessary developer features on production devices.

8.4 Outdated Android Version

ADB can identify the Android version and security patch level.

Risk: An outdated device may contain known vulnerabilities.

Protection: Keep Android and security patches up to date.

8.5 Excessive Debugging Information

Diagnostic commands may reveal system properties, services, packages, and configuration information.

Risk: Such information may assist an attacker during reconnaissance.

Protection: Restrict debugging access and avoid exposing diagnostic interfaces on production devices.

9. Security Analysis

The collected information can be analyzed using the following parameters:

Parameter	Purpose	Security Significance
Device Model	Identifies hardware	Helps determine device-specific security requirements

Android Version	Identifies OS version	Older versions may contain known vulnerabilities
Security Patch	Identifies update level	Indicates patching status
ADB Status	Checks debugging configuration	Enabled debugging increases attack surface
Developer Options	Checks development configuration	Unnecessary options should be disabled
ADB Connection State	Identifies connected/authorized devices	Unknown devices may indicate a security concern
Network Debugging	Checks network-based debugging	Exposure over networks increases risk

10. Security Best Practices

The following practices should be followed when using ADB:

1. Enable USB debugging only when required.
2. Connect only to trusted computers.
3. Do not accept unexpected ADB authorization requests.
4. Disable debugging features after testing.
5. Avoid using network-based ADB on untrusted networks.
6. Keep the Android operating system and security patches updated.
7. Avoid using development configurations on production devices.
8. Protect physical access to devices used for testing.
9. Use ADB only on devices for which proper authorization has been obtained.
10. Remove unnecessary debugging configurations before deploying an application or device.

11. Applications of ADB in Security Testing

ADB is useful in authorized Android security testing for:

- Device configuration analysis
- Application debugging
- Log analysis

- Security testing of applications
- Package and permission inspection
- Device troubleshooting
- Malware analysis in controlled environments
- Checking security configurations
- Testing application behavior on Android devices

12. Advantages of ADB

- Provides command-line access to Android devices.
- Allows efficient device information collection.
- Supports application debugging and testing.
- Provides access to Android diagnostic information.
- Can be used with both physical devices and emulators.
- Helps security professionals analyze device configurations.

13. Limitations

ADB is a powerful administrative and debugging interface, but its capabilities depend on the device's Android version, authorization state, security configuration, and available permissions. ADB access should therefore not be treated as unrestricted access to every Android device. Modern Android security mechanisms restrict many operations, particularly when the device is not rooted or the computer has not been authorized.

14. Result

The Android device information was collected using ADB commands, including device model, Android version, security patch level, battery information, and system properties. Debugging-related settings were examined to identify potentially exposed development features and understand their associated security risks.

15. Conclusion

ADB is an important tool for Android development, troubleshooting, and authorized security testing. It provides a convenient interface for collecting device information and examining system configurations. However, debugging interfaces such as USB or network-based ADB can increase the attack surface when unnecessarily enabled or improperly protected.

Therefore, ADB should be used only when required, debugging access should be limited to trusted systems, and unnecessary developer and debugging features should be disabled on production devices. Regular security updates and proper device configuration further reduce the risks associated with exposed debugging functionality.

